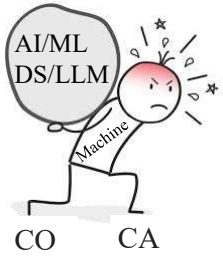
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)



Lecture #17

08 April 2026

Ashoka University

BBB

Review

Q1. What is the maximum speed-up achievable in MIPS pipeline and why?

IF → ID/RR → EX → Mem → WB

Ans. Standard MIPS supports a 5-stage pipeline;
Also, Max Speed-up = # pipeline stages;

Q2. Which factors prevent the realization of full pipeline speed?

Ans.
1. Pipeline overhead;
2. Hazards;

Review

Q. In today's processors such as ARM Cortex A-Series or X-Series, what is the typical depth of the pipeline?

Ans. Earlier version of ARM processors such as ARM9 (1997) comprises the classical 5-stage pipeline;

Modern ARM processors (2020+) support ~ 7 – 15 stages of pipeline architecture to achieve high speed-up; ARM Cortex-X4 (2023) – 10-stage pipeline;

Classical 5-stage pipelining is still being deployed in many microcontrollers and embedded systems (ARM, NXP, RISC-V, etc;)

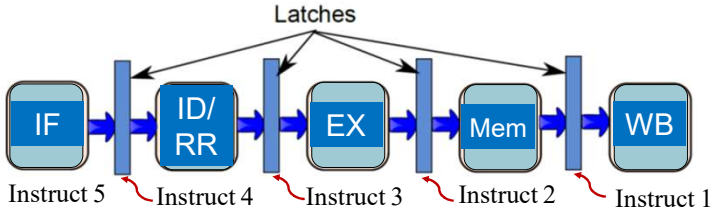
Review

Q. Why inter-stage latches are needed for pipeline implementation?

IF → ID/RR → EX → Mem → WB

MIPS Pipeline

- Pipeline registers (latches) are inserted between two consecutive pipeline stages
- All latches are synchronized with the system clock and seamlessly transfer instruction-bits and other intermediate data forward, across consecutive pipeline stages



Status during the fifth clock pulse

Review: Data Hazards in Pipelining

- A *hazard* refers to the possibility of erroneous execution of an instruction in a pipeline
- A *data hazard* represents the possibility of erroneous execution because of the unavailability of correct data

Pipelining: Data Hazards

```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

$s0 updated in the 5th clock cycle;

cycle	1	2	3	4	5	6
add	IF	ID/RR	EX	MEM	WB	
sub		IF	ID/RR	EX	MEM	WB

$s0 read request in 3rd clock cycle

Solution

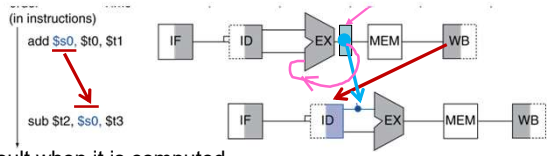
cycle	1	2	3	4	5	6	7	8	9
add	IF	ID/RR	EX	MEM	WB				
sub		✸	✸	✸	IF	ID/RR	EX	MEM	WB

✸ Bubble (Stall): Pipeline interlocking
CPU-time increases; Can we do better??

Improved hardware solution to fix data hazard: Forwarding (*a.k.a.* By-passing)

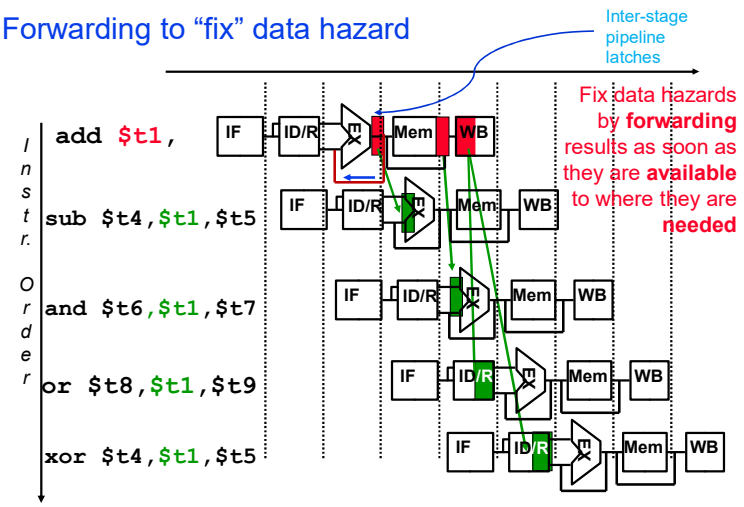
```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

cycle	1	2	3	4	5	6
add	IF	ID/RR	EX	MEM	WB	
sub		IF	ID/RR	EX	MEM	WB



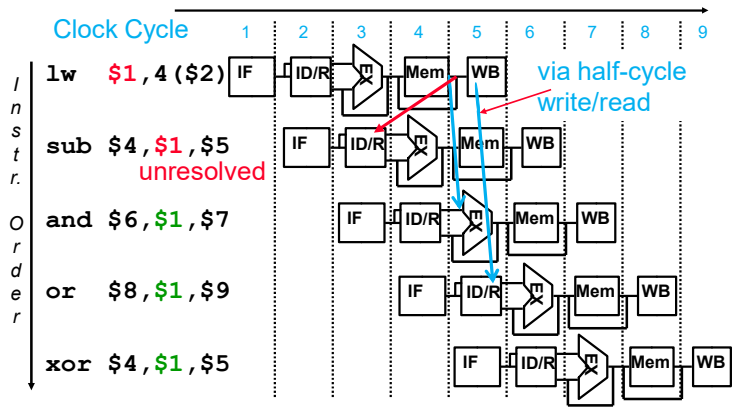
- Use result when it is computed
  - Don't wait for it to be stored in a register
  - Additional datapath needed from the output of ALU (EX) to the input of the same ALU!

Forwarding to "fix" data hazard



Data forwarding (in time); data-path set backward (in space);
No stalls are needed (saving two)!

Forwarding with Load-Use Data Hazards



So, we need to insert additional datapath backward (in space) from Mem/WB latch to the input of EX (Mem → EX Forwarding in time)

Example: Data Hazards in Pipeline

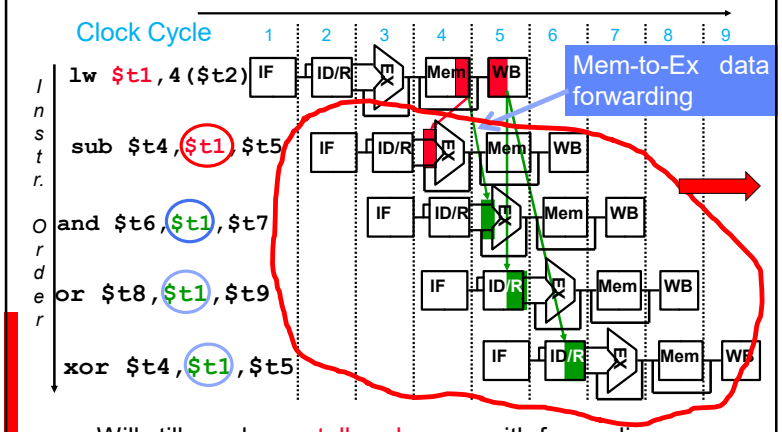
CC	1	2	3	4	5	6	7	8
ADD R1, R2, R3	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

data hazard

With forwarding mechanism such as EX → EX and Mem → EX all data hazards in this cases can be eliminated.

Total # clock cycles to run this code with forwarding = 8
Suppose, there is no data forwarding; then # clock cycles = 10

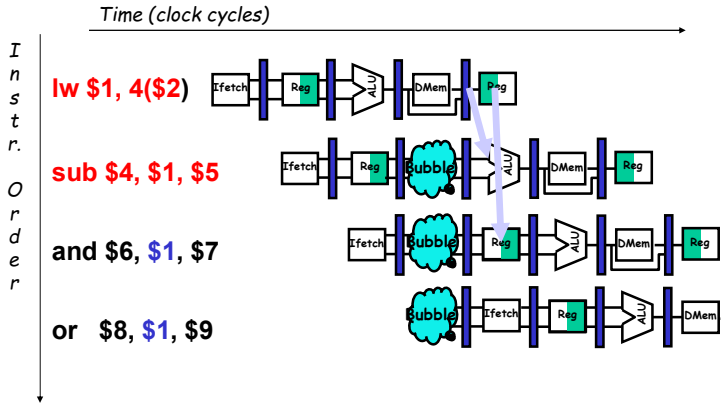
Forwarding with Load-Use Data Hazards



- Will still need one stall cycle even with forwarding
- We cannot forward data "backward in time"!

Data Hazards

The stall is necessary as shown here.



There are some instances where hazards occur, even with forwarding.

Example: Load-Use Data Hazards in Pipeline

CC	1	2	3	4	5	6	7	8
LW R1, 0(R2)	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

data hazard

stall-cycle = 1

CC	1	2	3	4	5	6	7	8	9
LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

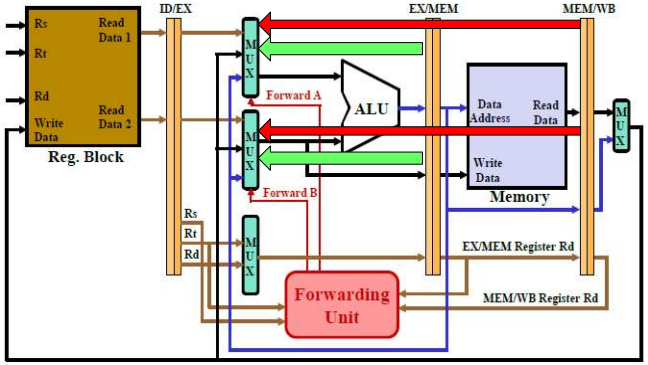
Total # clock cycles to run this code = 9

Stall is inevitable when the *destination* of load is a *source* of the next instruction

Summary: Load-Use Data Hazard

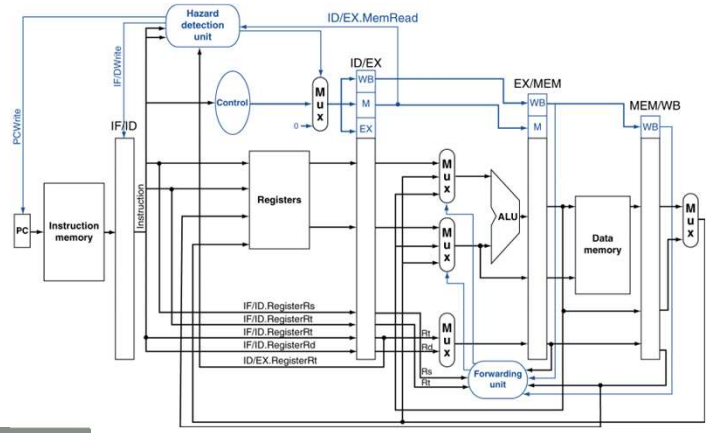
- Can't always avoid stalls by forwarding
  - If value not computed when needed
  - Can't forward backward in time!
  - Need stalls so that forwarding is possible

Data Forwarding Needs Extra Hardware
Forwarding EX → EX and Mem → EX



We insert additional datapath backward (in space) from both EX/Mem-latch and Mem/WB-latch to the input of EX (EX → EX, and Mem → EX for data forwarding in time)

Datapath with Forwarding and Hazard Detection



Tabular Representation of Pipeline

LW R1, 4(R2)	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

Resolving data hazard

LW R1, 4(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

Inserting vertical bubbles is equivalent to inserting horizontal bubbles

Data hazards – Software Interlocking

- C code for `A = B + E; C = B + F;`

Reorder code to avoid use of load result in the next instruction

lw $t1, 0($t0)	IF	ID	EX	MEM	WB
add $t3, $t1, $t1		IF	ID	EX	MEM	WB

```
lw  $t1, 0($t0)
lw  $t2, 4($t0)
add $t3, $t1, $t2
sw  $t3, 12($t0)
lw  $t4, 8($t0)
add $t5, $t1, $t4
sw  $t5, 16($t0)
```

stall

stall

2 stalls inevitable even with forwarding; Total = 13 cycles

```
lw  $t1, 0($t0)
lw  $t2, 4($t0)
lw  $t4, 8($t0)
add $t3, $t1, $t2
sw  $t3, 12($t0)
add $t5, $t1, $t4
sw  $t5, 16($t0)
```

No stall; Total = 11 cycles with forwarding; for load-use cases, insert *NOP*

Software Solution

RAW Data Hazards

Read-After-Write (RAW)

$Instr_J$ tries to read operand before $Instr_I$ writes it

```
I: add $t1,$t2,$t3
J: sub $t4,$t1,$t3
```

- Caused by "dependence"

WAW Data Hazards

ϕ: stall

Write-After-Write (WAW)

Because of WAW hazard, *sub* tries to write $t1 *before* it is written by *lw*

```
I:  lw $t1,0($t2)
J: add $t4, $t1, $t2
K: sub $t1,$t2,$t3
```

data cache miss

$t1 is written by *lw* in 8th clock cycle

RAW hazard

I:	IF	ID/RR	EX	MEM	ϕ	ϕ	ϕ	WB
J:		IF	ID/RR	EX	MEM	WB		
K:			IF	ID/RR	EX	MEM	WB	

$t1 is written by *sub* in 7th clock cycle (WAW)

Write in $t1 is performed in wrong order!

WAR Data Hazards

Write-After-Read (WAR)

```
I: add $t4,$t1,$t3
J: sub $t1,$t2,$t3
```

Can Instr J write to register $t1 earlier before Instr I reads it?

I:	IF	ID/RR	EX	MEM	WB	
K:		IF	ID/RR	EX	MEM	WB

In the MIPS in-order pipeline scheme, reads are early (ID/RR) and writes are late (WB); hence, WAR cannot occur.

When can WAR occur?

Out-of-order execution; unbalanced pipeline such as that for FP

Read-After-Read (RAR): Not a hazard

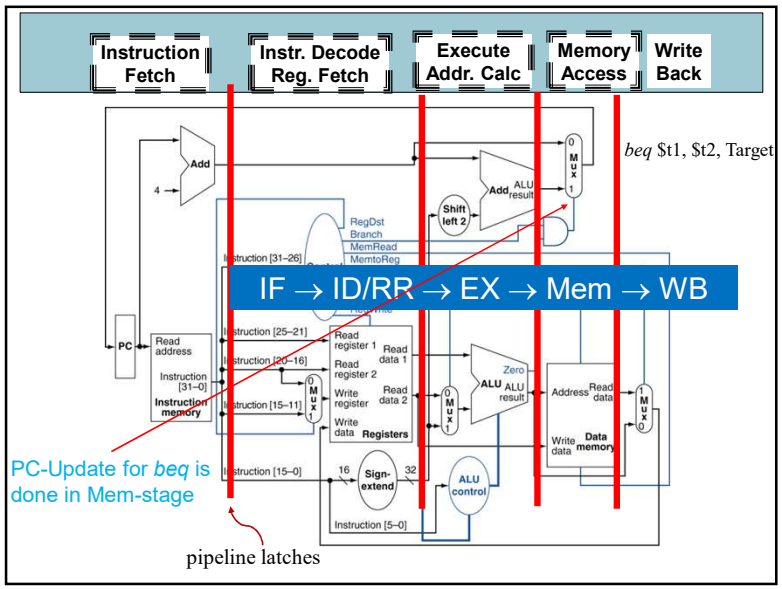
Control Hazards

A control hazard arises when the computation of the destination of a branch and update of PC are awaited, and hence, the instruction to be executed next, is uncertain at this moment

Let $t1 = $t2

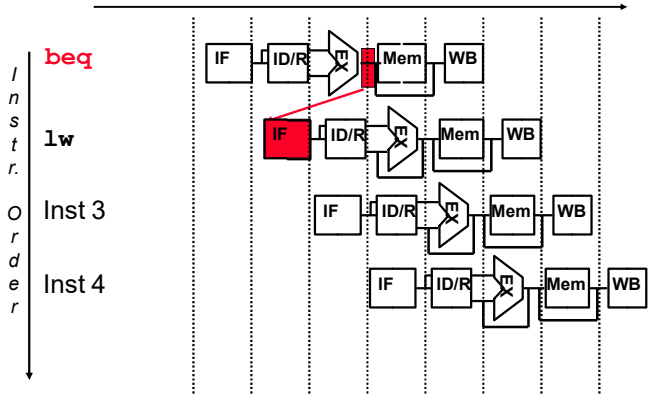
beq $t1, $t2, Target	IF	ID/RR	EX	MEM	WB
lw $t3, 0($t4)		IF	ID/RR	EX	MEM	WB
…					
…					
Target: add $t1, $t2, $t3					

What will happen to the *lw* instruction?



Example: Control Hazards due to Branch

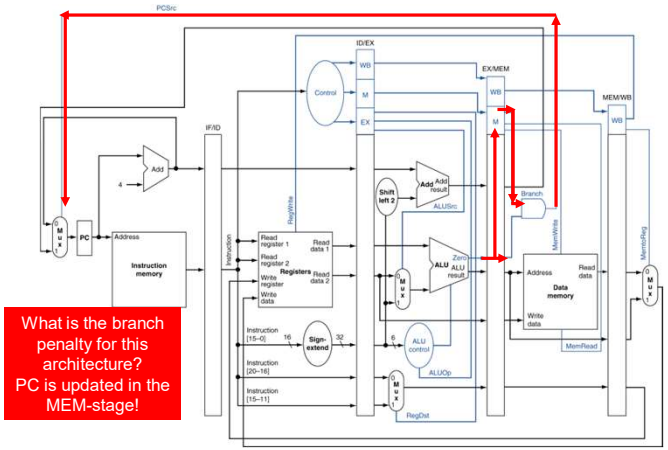
- Dependencies backward in time cause hazards



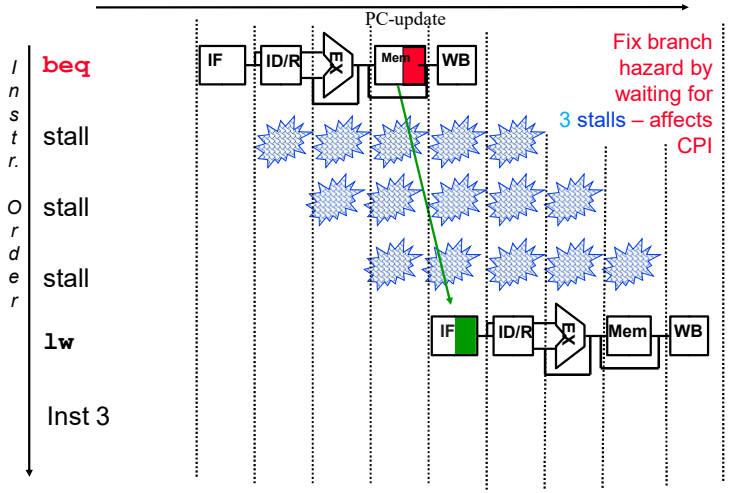
Branch Penalty

beq	IF	ID	EX	MEM	WB
	*	*	*	IF	ID	EX	MEM	WB
lw					

branch penalty: 3 clock cycles

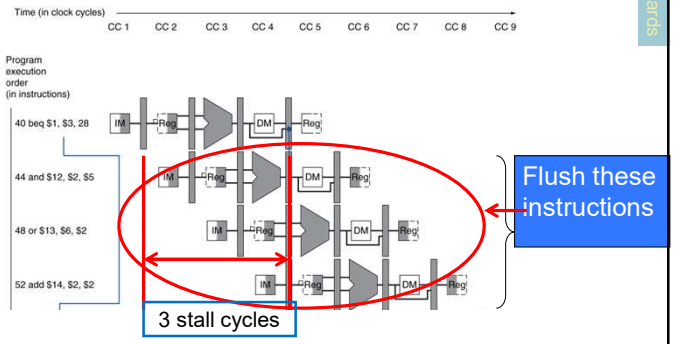


One Way to "Fix" a Control Hazard: Insert stalls



Control Hazards: Continue, flush if branch is taken

- If branch outcome determined in MEM

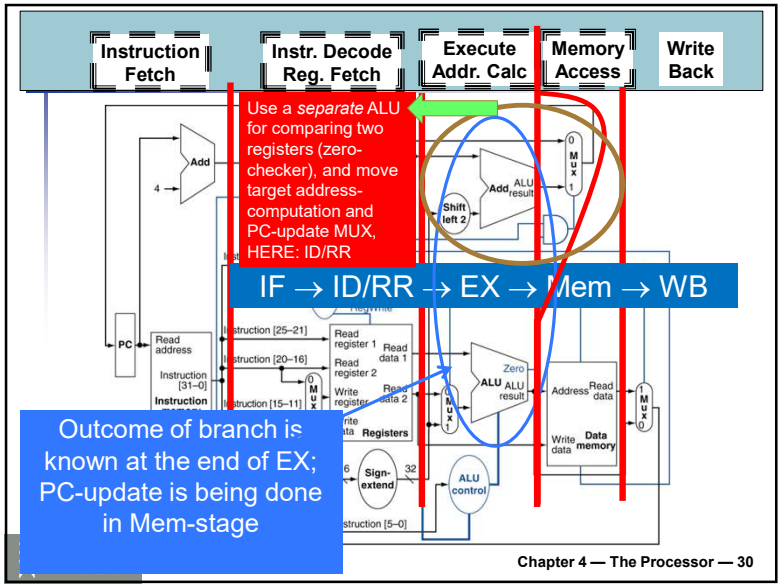


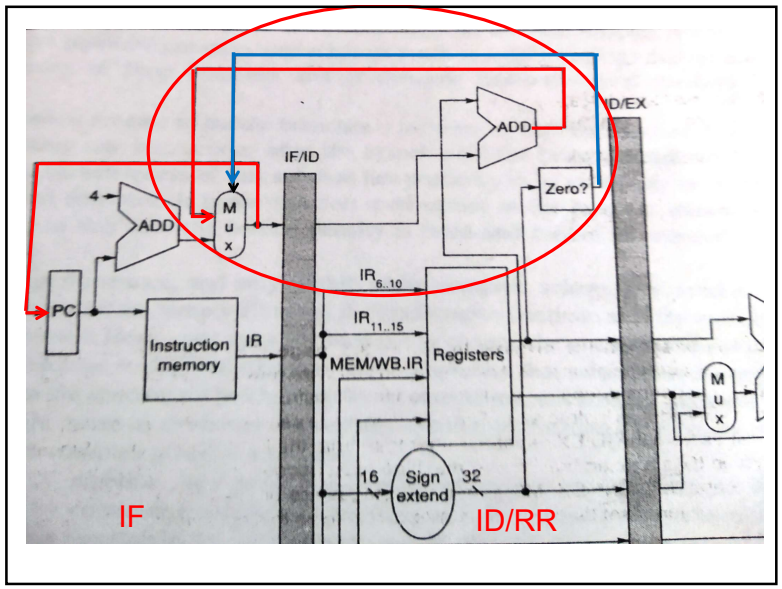
Control Hazards

- Hardware solution?

Can we compute branch target and PC-Update, earlier in the pipeline?

Would reduce stalls for successor instructions!



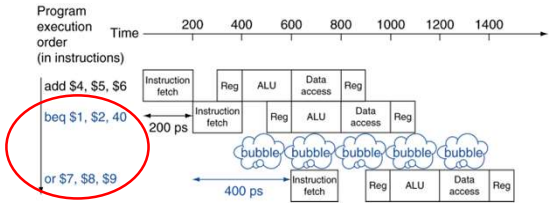


Control Hazards (*beq*)

- In MIPS pipeline, we can reduce branch penalty
  - by comparing registers (zero-checking) and computing target (PC-update) early in the pipeline
  - Add hardware to do it earlier in ID/RR-stage

Stall on Branch

- Wait until branch outcome determined (at ID/RR-stage) before fetching next instruction



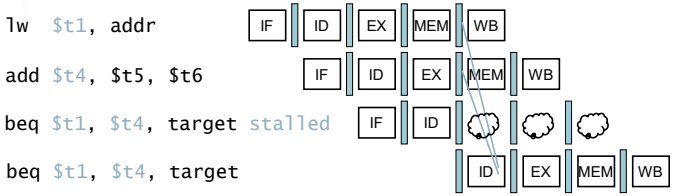
Instead of "three stalls", we now have only "one stall"

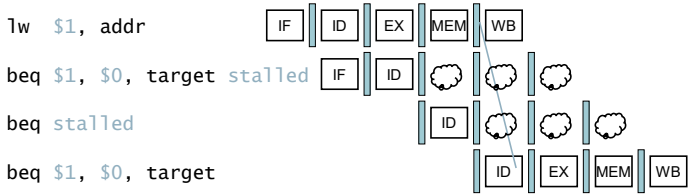
Stall on Branch

- Moving branch-target determination from Mem to ID/RR-stage helps resolve hazards in successor instructions
- However,
- it may pose problems to *preceding* instructions, when a branch source-register is a destination register of a preceding instruction!

Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
  - Need 1 stall cycle



Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
  - Need 2 stall cycles